



## 密码学引论实验报告

姓名: 江李阳, 王健一, 邱泽辉

学号: 202400460042, 202400460037, 202400460046

专业: 密码科学与技术

日期: May 13, 2026

## Contents

|          |                                    |          |
|----------|------------------------------------|----------|
| <b>1</b> | <b>实验一</b>                         | <b>1</b> |
| 1.1      | 实验目的                               | 1        |
| 1.2      | 实验原理                               | 1        |
| 1.2.1    | 差分分布表定义                            | 1        |
| 1.3      | 实验数据分析                             | 1        |
| 1.3.1    | 差分分布表 (DDT) 概览                     | 1        |
| 1.3.2    | 高概率差分路径                            | 1        |
| 1.3.3    | 概率传播种类统计                           | 2        |
| 1.3.4    | 差分碰撞分析 (输入非 0 输出为 0)               | 2        |
| 1.4      | $S_1$ 盒的实验结果及分析                    | 3        |
| 1.5      | $S_2$ 盒实验结果分析                      | 3        |
| 1.6      | $S_3$ 盒实验结果分析                      | 4        |
| 1.7      | 实验结论                               | 5        |
| <b>2</b> | <b>实验二</b>                         | <b>5</b> |
| 2.1      | 实验目的                               | 5        |
| 2.2      | 实验原理                               | 6        |
| 2.2.1    | 积分特性与 $\Delta$ -集合                 | 6        |
| 2.2.2    | AddRoundKey 不影响活跃字节                | 6        |
| 2.2.3    | SubBytes 不影响活跃字节                   | 7        |
| 2.2.4    | ShiftRows 不影响活跃字节数量, 但是可能影响活跃字节的位置 | 7        |
| 2.2.5    | 列混淆 (MixColumns) 的作用               | 7        |
| 2.2.6    | 第一轮结束结果                            | 8        |
| 2.2.7    | 第二轮结束结果                            | 8        |
| 2.2.8    | 第三轮结束结果                            | 9        |
| 2.3      | 现象分析与讨论                            | 9        |
| 2.3.1    | 3 轮与 4 轮现象分析                       | 9        |
| 2.3.2    | 删除 MixColumns 的影响                  | 10       |
| 2.4      | 实验结论                               | 10       |
| 2.5      | 实验过程与结果                            | 10       |
| 2.5.1    | 实验设置                               | 10       |
| 2.5.2    | 结果记录                               | 10       |
| .1       | 附录: 完整 $S_1$ 盒差分分布表 (DDT)          | 11       |
| .2       | 附录: 完整 $S_2$ 盒差分分布表 (DDT)          | 12       |
| .3       | 附录: 完整 $S_3$ 盒差分分布表 (DDT)          | 14       |
| .4       | 附录                                 | 15       |

# 1 实验一

## 1.1 实验目的

1. 理解分组密码加密算法 DES 中 S 盒的非线性混淆原理。
2. 掌握差分分布表的构造方法。

## 1.2 实验原理

### 1.2.1 差分分布表定义

对于  $6 \rightarrow 4$  位的 S 盒，输入差分  $\alpha$  与输出差分  $\beta$  的关联强度由  $N_S(\alpha, \beta)$  衡量。其数学定义为：在一个输入空间内，满足特定输入差分导致特定输出差分的输入值数量。

$$N_S(\alpha, \beta) = \#IN_S(\alpha, \beta) = \{X \in \{0, 1\}^6 \mid S(X) \oplus S(X \oplus \alpha) = \beta\}$$

其中  $X$  遍历所有  $2^6 = 64$  种输入可能，最终构造出  $2^6 \times 2^4$  的差分分布表。

## 1.3 实验数据分析

本次实验用了 S1, S2, S3 三组 S 盒，下面以 S1 为例进行分析，代码运行结果图如下

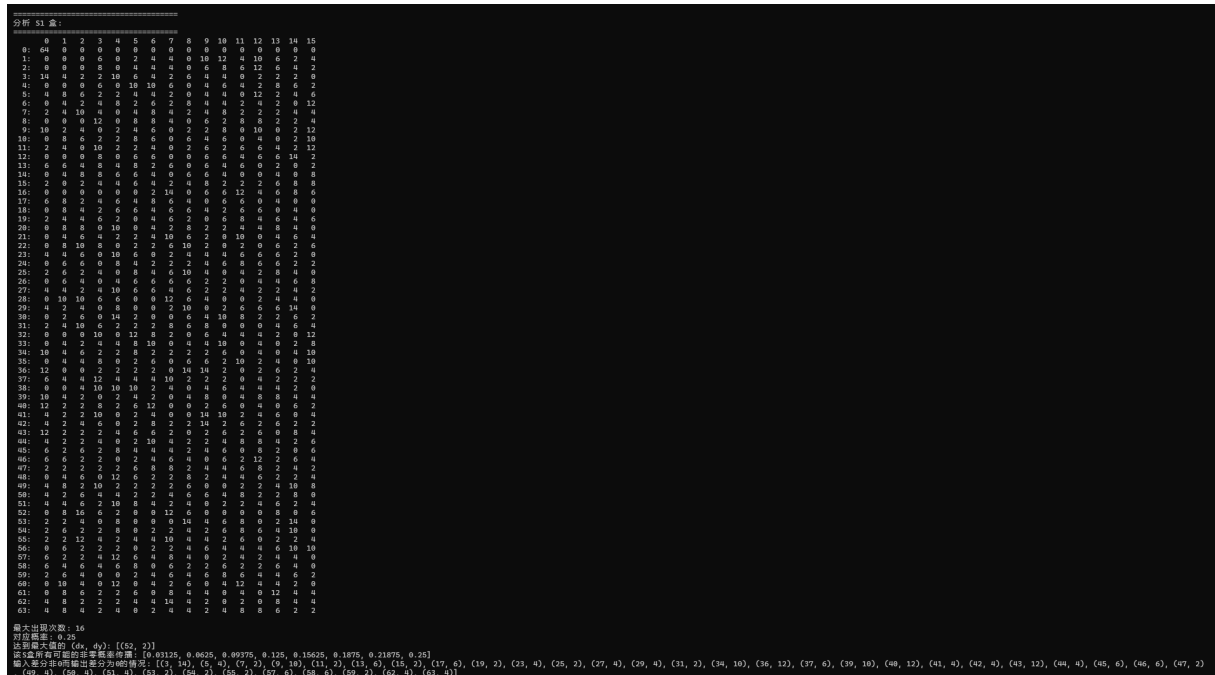


Figure 1: 实验一代码运行部分结果图

### 1.3.1 差分分布表 (DDT) 概览

本实验生成的 S1 盒 DDT 规模为  $64 \times 16$ 。表中的数值反映了特定差分传播的频数。具体差分分布表见附录 A。

### 1.3.2 高概率差分路径

实验数据如下表所示：

Table 1: S1 盒高概率差分路径汇总表

| 排名 | 计数 | 概率       | (输入差分, 输出差分) 列表                                                                                                                                                                                                                                                                                                                       |
|----|----|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | 64 | 1.000000 | (0x00, 0x0)                                                                                                                                                                                                                                                                                                                           |
| 2  | 16 | 0.250000 | (0x34, 0x2)                                                                                                                                                                                                                                                                                                                           |
| 3  | 14 | 0.218750 | (0x03, 0x0), (0x0C, 0xE), (0x10, 0x7), (0x1D, 0xE),<br>(0x1E, 0x4), (0x24, 0x8), (0x24, 0x9), (0x29, 0x9),<br>(0x2A, 0x9), (0x35, 0x8), (0x35, 0xE), (0x3E, 0x7)                                                                                                                                                                      |
| 4  | 12 | 0.187500 | (0x01, 0xA), (0x02, 0xC), (0x05, 0xC), (0x06, 0xF),<br>(0x08, 0x3), (0x09, 0xF), (0x0B, 0xF), (0x10, 0xB),<br>(0x1C, 0x7), (0x20, 0x5), (0x20, 0xF), (0x24, 0x0),<br>(0x25, 0x3), (0x28, 0x0), (0x28, 0x6), (0x2B, 0x0),<br>(0x2E, 0xC), (0x30, 0x4), (0x34, 0x7), (0x37, 0x2),<br>(0x39, 0x4), (0x3C, 0x4), (0x3C, 0xB), (0x3D, 0xD) |

### 1.3.3 概率传播种类统计

实验统计显示，该 S 盒共有 9 种不同的非零计数值，对应 9 种传播概率：

- 低概率路径：计数为 2, 4, 6，对应的最低传播概率为 0.0312。
- 中概率路径：计数为 8, 10, 12。
- 高概率路径：计数为 14, 16，最高传播概率达到 0.2500（即 16/64）。

### 1.3.4 差分碰撞分析（输入非 0 输出为 0）

实验结果表明，当输入差分  $\alpha \neq 0$  时，输出差分  $\beta$  可能为 0。原因分析：DES 的 S 盒是一个从 6 位到 4 位的压缩映射。根据抽屉原理（Pigeonhole Principle），必然存在多个不同的输入映射到相同的输出。即：

$$\exists X_1 \neq X_2, \text{ s.t. } S(X_1) = S(X_2) \implies S(X_1) \oplus S(X_2) = 0$$

当  $\alpha = X_1 \oplus X_2$  时，该输入差分会导致输出差分为 0。这在差分分析中意味着该 S 盒在该路径下是“不活动的”。

Table 3: S<sub>1</sub> 盒差分碰撞分布表 ( $\alpha \neq 0, \beta = 0$ )

| 输入差分 | 碰撞频数 | 输入差分 | 碰撞频数 | 输入差分 | 碰撞频数 |
|------|------|------|------|------|------|
| 0x03 | 14   | 0x25 | 6    | 0x2C | 4    |
| 0x24 | 12   | 0x2D | 6    | 0x31 | 4    |
| 0x28 | 12   | 0x2E | 6    | 0x32 | 4    |
| 0x2B | 12   | 0x39 | 6    | 0x33 | 4    |
| 0x09 | 10   | 0x3A | 6    | 0x3E | 4    |
| 0x22 | 10   | 0x05 | 4    | 0x3F | 4    |
| 0x27 | 10   | 0x17 | 4    | 0x07 | 2    |
| 0x0D | 6    | 0x1B | 4    | 0x0B | 2    |
| 0x11 | 6    | 0x1D | 4    | 0x0F | 2    |
| ...  | ...  | 0x29 | 4    | ...  | ...  |

#### 1.4 S<sub>1</sub> 盒的实验结果及分析

根据程序计算得到的 S<sub>1</sub> 盒差分分布表可知，其最大差分传播次数为 16，因此最大差分传播概率为

$$P_{\max} = \frac{16}{64} = 0.25.$$

进一步统计发现，达到该最大概率的差分对为  $(\Delta x, \Delta y) = (52, 2)$ 。此外，对差分分布表中所有非零项对应的传播概率进行整理可得，S<sub>1</sub> 盒共具有 8 种非零差分传播概率，分别为

0.03125, 0.0625, 0.09375, 0.125, 0.15625, 0.1875, 0.21875, 0.25.

这说明 S<sub>1</sub> 盒的差分传播分布较为分散，避免了大量高概率差分路径的集中出现，从而增强了其抗差分密码分析的能力。

Table 4: S<sub>1</sub> 盒差分分布表的主要统计结果

| 统计指标                              | 结果      |
|-----------------------------------|---------|
| 最大差分传播次数                          | 16      |
| 最大差分传播概率                          | 0.25    |
| 达到最大概率的差分对 $(\Delta x, \Delta y)$ | (52, 2) |
| 非零差分传播概率种类数                       | 8       |

Table 5: S<sub>1</sub> 盒差分分布表的部分节选

| $\Delta x$ | 0  | 1 | 2  | 3 | 4 | 5 | 6 | 7  | 8 | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|------------|----|---|----|---|---|---|---|----|---|----|----|----|----|----|----|----|
| 0          | 64 | 0 | 0  | 0 | 0 | 0 | 0 | 0  | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 1          | 0  | 0 | 0  | 6 | 0 | 2 | 4 | 4  | 0 | 10 | 12 | 4  | 10 | 6  | 2  | 4  |
| 52         | 0  | 8 | 16 | 6 | 2 | 0 | 0 | 12 | 6 | 0  | 0  | 0  | 8  | 0  | 0  | 6  |

#### 1.5 S<sub>2</sub> 盒实验结果分析

根据程序计算得到的 S<sub>2</sub> 盒差分分布表可知，其最大差分传播次数为 16，因此最大差分传播概率为

$$P_{\max} = \frac{16}{64} = 0.25.$$

进一步统计发现，达到该最大概率的差分对共有 5 个，分别为

$$(8, 10), (29, 7), (54, 13), (59, 0), (61, 1).$$

对差分分布表中所有非零项对应的传播概率进行整理可得， $S_2$  盒共具有 8 种非零差分传播概率，分别为

$$0.03125, 0.0625, 0.09375, 0.125, 0.15625, 0.1875, 0.21875, 0.25.$$

这说明  $S_2$  盒的差分传播概率分布较为分散，并未集中在极少数高概率差分路径上，从而有助于增强其抗差分密码分析的能力。

此外， $S_2$  盒中达到最大概率的差分对数量多于  $S_1$  盒，表明不同  $S$  盒在差分传播结构上存在差异；这也说明在 DES 的整体设计中，各个  $S$  盒通过不同的差分特征共同提升了系统的安全性。

Table 6:  $S_2$  盒差分分布表的主要统计结果

| 统计指标                              | 结果                                             |
|-----------------------------------|------------------------------------------------|
| 最大差分传播次数                          | 16                                             |
| 最大差分传播概率                          | 0.25                                           |
| 达到最大概率的差分对 $(\Delta x, \Delta y)$ | $(8, 10), (29, 7), (54, 13), (59, 0), (61, 1)$ |
| 非零差分传播概率种类数                       | 8                                              |

Table 7:  $S_2$  盒差分分布表的部分节选

| $\Delta x$ | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|------------|----|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|
| 0          | 64 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 1          | 0  | 0 | 0 | 4 | 0 | 2 | 6 | 4 | 0 | 14 | 8  | 6  | 8  | 4  | 6  | 2  |
| 8          | 0  | 0 | 0 | 4 | 0 | 4 | 0 | 8 | 0 | 10 | 16 | 6  | 6  | 0  | 6  | 4  |

## 1.6 $S_3$ 盒实验结果分析

根据程序计算得到的  $S_3$  盒差分分布表可知，其最大差分传播次数同样为 16，因此最大差分传播概率为

$$P_{\max} = \frac{16}{64} = 0.25.$$

进一步统计发现，达到该最大概率的差分对共有 4 个，分别为

$$(32, 13), (39, 3), (45, 7), (52, 4).$$

对差分分布表中所有非零项对应的传播概率进行整理可得， $S_3$  盒同样具有 8 种非零差分传播概率，分别为

$$0.03125, 0.0625, 0.09375, 0.125, 0.15625, 0.1875, 0.21875, 0.25.$$

这表明  $S_3$  盒的差分传播规律与  $S_1$ 、 $S_2$  盒总体一致，即最大差分传播概率受到控制，且概率分布较为分散。

值得注意的是， $S_3$  盒达到最大概率的差分对数量少于  $S_2$  盒，但多于  $S_1$  盒，这进一步反映出不同  $S$  盒在差分特性上的差异。总体来看， $S_3$  盒同样满足 DES 对抗差分分析的设计要求。

Table 8:  $S_3$  盒差分分布表的主要统计结果

| 统计指标                              | 结果                                    |
|-----------------------------------|---------------------------------------|
| 最大差分传播次数                          | 16                                    |
| 最大差分传播概率                          | 0.25                                  |
| 达到最大概率的差分对 $(\Delta x, \Delta y)$ | $(32, 13), (39, 3), (45, 7), (52, 4)$ |
| 非零差分传播概率种类数                       | 8                                     |

Table 9:  $S_3$  盒差分分布表的部分节选

| $\Delta x$ | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8 | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|------------|----|---|---|---|---|---|---|----|---|----|----|----|----|----|----|----|
| 0          | 64 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 1          | 0  | 0 | 0 | 2 | 0 | 4 | 2 | 12 | 0 | 14 | 0  | 4  | 8  | 2  | 6  | 10 |
| 32         | 0  | 0 | 0 | 0 | 0 | 4 | 4 | 8  | 0 | 2  | 2  | 4  | 10 | 16 | 12 | 2  |

## 1.7 实验结论

本实验利用 Python 程序对 DES 的三个 S 盒  $S_1, S_2, S_3$  进行了差分分布分析，成功生成了对应的差分分布表，并统计了其安全性指标。实验结果表明：

1. 三个 S 盒的最大差分传播概率均为 0.25；
2. 三个 S 盒均具有 8 种非零差分传播概率；
3. 不同 S 盒达到最大概率的差分对数量不同，说明其差分结构存在差异；
4. 当输入差分非零时，输出差分仍可能为零，这是由 S 盒的非单射性质决定的。

综合来看，DES 的 S 盒通过精心设计，使得差分分布尽量分散，避免出现过高的差分传播概率，从而增强了 DES 对差分密码分析的抵抗能力。本实验通过对  $S_1, S_2, S_3$  的具体统计验证了这一点，也说明差分分布表是评估分组密码非线性部件安全性的重要工具。

通过本实验，进一步理解了差分密码分析的基本原理以及 DES S 盒在抗差分攻击设计中的重要作用。

## 2 实验二

### 2.1 实验目的

1. 验证 AES 算法在 3 轮加密下的平衡性（Balance）现象。
2. 探究轮数增加对积分特性消失的影响。
3. 分析列混淆变换（MixColumns）在密码扩散中的核心作用。

### 2.2 实验设计与流程

#### 2.2.1 明文集合构造

构造 3 组特殊的明文集合  $\mathcal{P} = \{P_0, P_1, \dots, P_{255}\}$ 。每组集合中，选定一个字节位置  $i$ （图中标红位置）作为活跃字节，使其取遍  $0x00$  到  $0xFF$ ：

$$P_j^{(i)} = j, \quad j \in \{0, 1, \dots, 255\} \quad (1)$$

其余 15 个字节位置固定为常数  $C$ 。

#### 2.2.2 实验操作步骤

1. 加密阶段：选择随机密钥  $K$ ，对每个明文集合执行 3 轮完整的 AES 加密（包含 SubBytes, ShiftRows, MixColumns, AddRoundKey）。
2. 求和阶段：对每组得到的 256 个密文在  $GF(2^8)$  空间内进行异或（XOR）求和：

$$S = \bigoplus_{j=0}^{255} C_j \quad (2)$$

3. 对比实验：分别测试 4 轮加密情况，以及删除列混合（MixColumns）变换后的情况。



## 2.3 实验过程与结果

### 2.3.1 实验设置

构造 3 组  $\Lambda$ -集合, 每组包含 256 个明文, 在矩阵坐标 (1,0) 处遍历。分别测试 3 轮、4 轮以及无 MixColumns 情况下的异或和。我们选取的明文如下:

$$P_0 = \begin{bmatrix} 16 & 34 & 52 & 70 \\ * & 106 & 124 & 142 \\ 145 & 163 & 181 & 199 \\ 217 & 235 & 253 & 15 \end{bmatrix}$$

$$P_1 = \begin{bmatrix} 1 & 19 & 37 & 55 \\ * & 91 & 109 & 127 \\ 128 & 146 & 164 & 182 \\ 200 & 218 & 236 & 254 \end{bmatrix}$$

$$P_2 = \begin{bmatrix} 255 & 238 & 221 & 204 \\ * & 170 & 153 & 136 \\ 119 & 102 & 85 & 68 \\ 51 & 34 & 17 & 0 \end{bmatrix}$$

我们选择的密钥如下:

key = 0xabf266f7a189b08e5db4a56ea585ae0f

### 2.3.2 结果记录

保持 MC 变换, 代码运行结果如下

```

PS D:\DeskTop\Crypto_Introduction\crypto_introduction\first_experiment(JLY)> python .\task2.py
=====
加密密钥为b'\xab\xf2\xf7\xa1\x89\xb0\x8e'\xb4\xa5\xa5\x85\xae\x0f'
加密轮数3,是否MC变换True
第1组异或结果:[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第2组异或结果:[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第3组异或结果:[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
=====
加密密钥为b'\xab\xf2\xf7\xa1\x89\xb0\x8e'\xb4\xa5\xa5\x85\xae\x0f'
加密轮数4,是否MC变换True
第1组异或结果:[1, 148, 210, 196, 50, 25, 180, 204, 105, 68, 45, 146, 148, 54, 98, 54]
第2组异或结果:[212, 203, 214, 111, 67, 224, 161, 210, 0, 44, 22, 115, 43, 111, 43, 41]
第3组异或结果:[142, 66, 255, 232, 236, 26, 156, 81, 30, 102, 14, 80, 117, 2, 130, 224]
PS D:\DeskTop\Crypto_Introduction\crypto_introduction\first_experiment(JLY)>

```

Figure 2: 3 轮和 4 轮 AES 加密、结果异或 (保持 MC 变换)

可以看到当 AES 加密轮数为 3 组时, 这 3 个实验组变动位遍历 0 255 的每组总计 256 种密文加密后的异或结果均为 0, 但当 AES 加密轮数变为 4 轮后异或结果就变得各不相同。

删去 MC 变换, 代码运行结果如下

```

PS D:\DeskTop\Crypto_Introduction\crypto_introduction\first_experiment(JLY)> python .\task2.py
=====
加密密钥为 b'\xab\xf2\xf7\xa1\x89\xb0\x8e'\xb4\xa5\xa5\x85\xae\x0f'
加密轮数 3, 是否 MC 变换 False
第 1 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 2 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 3 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
=====
加密密钥为 b'\xab\xf2\xf7\xa1\x89\xb0\x8e'\xb4\xa5\xa5\x85\xae\x0f'
加密轮数 4, 是否 MC 变换 False
第 1 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 2 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 3 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
=====
加密密钥为 b'\xab\xf2\xf7\xa1\x89\xb0\x8e'\xb4\xa5\xa5\x85\xae\x0f'
加密轮数 5, 是否 MC 变换 False
第 1 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 2 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 3 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
=====
加密密钥为 b'\xab\xf2\xf7\xa1\x89\xb0\x8e'\xb4\xa5\xa5\x85\xae\x0f'
加密轮数 6, 是否 MC 变换 False
第 1 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 2 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 3 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
=====
加密密钥为 b'\xab\xf2\xf7\xa1\x89\xb0\x8e'\xb4\xa5\xa5\x85\xae\x0f'
加密轮数 7, 是否 MC 变换 False
第 1 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 2 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 3 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
=====
加密密钥为 b'\xab\xf2\xf7\xa1\x89\xb0\x8e'\xb4\xa5\xa5\x85\xae\x0f'
加密轮数 8, 是否 MC 变换 False
第 1 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 2 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 3 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
=====
加密密钥为 b'\xab\xf2\xf7\xa1\x89\xb0\x8e'\xb4\xa5\xa5\x85\xae\x0f'
加密轮数 9, 是否 MC 变换 False
第 1 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 2 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 3 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
=====
加密密钥为 b'\xab\xf2\xf7\xa1\x89\xb0\x8e'\xb4\xa5\xa5\x85\xae\x0f'
加密轮数 10, 是否 MC 变换 False
第 1 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 2 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
第 3 组异或结果: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
PS D:\DeskTop\Crypto_Introduction\crypto_introduction\first_experiment(JLY)> |

```

Figure 3: 10 轮 AES 加密、结果异或 (删除 MC 变换)

当删除 MC 变换时, 相同数据和代码, 即使加密 10 轮, 各组的异或结果也都是 0

## 2.4 实验现象原理分析

### 2.4.1 积分特性与 $\Lambda$ -集合

在对 AES 的积分攻击中，我们构造一个特殊的明文集合，称为  $\Lambda$ -集合。在该集合中，选定的字节遍历  $0 \dots 255$ （称为 **Active** 字节），而其余字节保持固定（称为 **Constant** 字节）。[?, ?]<sup>1</sup>

设活跃字节遍历的 256 个取值为  $x_i (i = 1, 2, \dots, 256)$ ，满足

$$\bigoplus_{i=1}^{256} x_i = 0$$

这种性质可以称为平衡性

下面给出证明

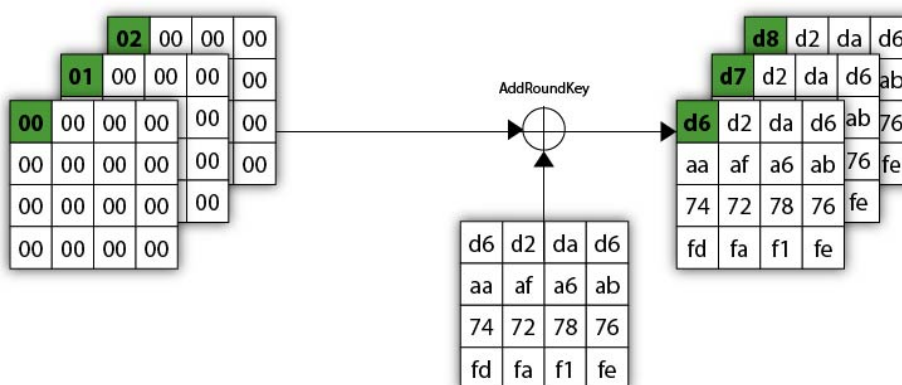
有限域  $GF(2^n)$  可以被看作是定义在  $GF(2)$  上的  $n$  维向量空间。这意味着域中的元素可以看作长度为  $n$  的二进制向量，考虑这  $2^n$  个向量在第  $i$  位上的取值。在所有可能的  $2^n$  个组合中，第  $i$  位为 0 的元素个数和为 1 的元素个数是相等的。具体来说，各有  $2^{n-1}$  个。因而有偶数个 1 在第  $i$  位进行异或，结果必然是 0。

实验表明，对于一个包含 256 个明文的  $\Lambda$ -集合，在经过 3 轮 AES 加密后，其密文状态满足：

$$\bigoplus_{i=1}^{256} C^{(i)} = 0 \quad (3)$$

下面展开具体说明

### 2.4.2 AddRoundKey 不影响活跃字节

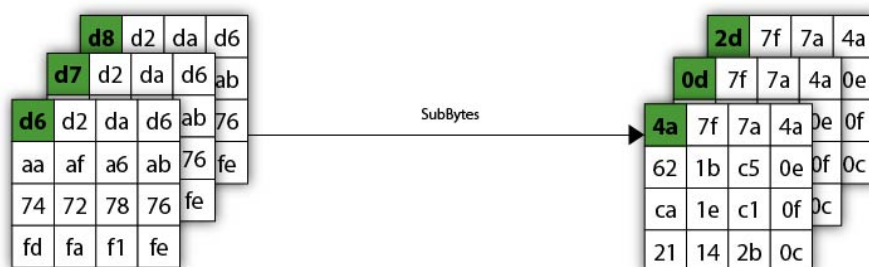


首先一开始先将明文和密钥进行 AddRoundKey 操作，之后进入第 1 轮。异或是线性运算，当活跃字节遍历  $0 \dots 255$  时，输出字节也会完整遍历  $0 \dots 255$ 。所以 AddRoundKey 操作不影响活跃性。<sup>2</sup>

<sup>1</sup>参考文献 2 的链接目前已失效 (或许是作者没有维护)，但是可以通过互联网档案馆 (Wayback Machine) 访问：<https://web.archive.org/web/20240101000000/https://www.davidwong.fr/blockbreakers/index.html>。

<sup>2</sup>实验中要求的活跃字节在矩阵坐标 (1,0) 处的字节，即第二行第一列的字节。此处引用的参考文献中活跃字节在位置 (0,0)(如图片所示)，此处引用文献图片为方便说明，实际原理相同。

### 2.4.3 SubBytes 不影响活跃字节

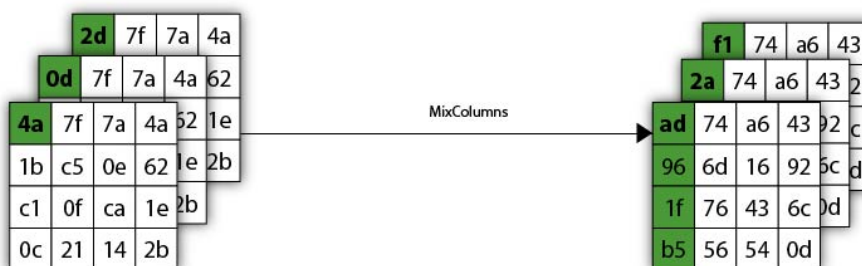


SubBytes(过 S 盒, 字节代换) 是一个  $\{0,1\}^8 \rightarrow \{0,1\}^8$  的映射。因此, 活跃字节在经过 SubBytes 后仍然保持活跃, 继续遍历  $0 \dots 255$ 。

### 2.4.4 ShiftRows 不影响活跃字节数量, 但是可能影响活跃字节的位置

ShiftRows 只是对状态矩阵的行进行循环移位, 不改变字节的值。因此, 活跃字节在经过 ShiftRows 后仍然保持活跃。由于第一行不进行移位, 所以文献中并未改变活跃字节位置。但是对于实验中的位置, 活跃字节在  $(1,0)$  的情况, 经过 ShiftRows 后它会移动到  $(1,3)$ , 但仍然是活跃的, 只是换了一个位置。

### 2.4.5 列混淆 (MixColumns) 的作用



MixColumns 是 AES 中唯一的线性扩散层。它负责在列内进行线性组合, 以列为单位, 每一列左乘一个矩阵。将输列混合运算的状态矩阵的每一列的 4 个字节以列向量表示, 记为

$$x = (x_0, x_1, x_2, x_3)^T$$

输出的列向量记为

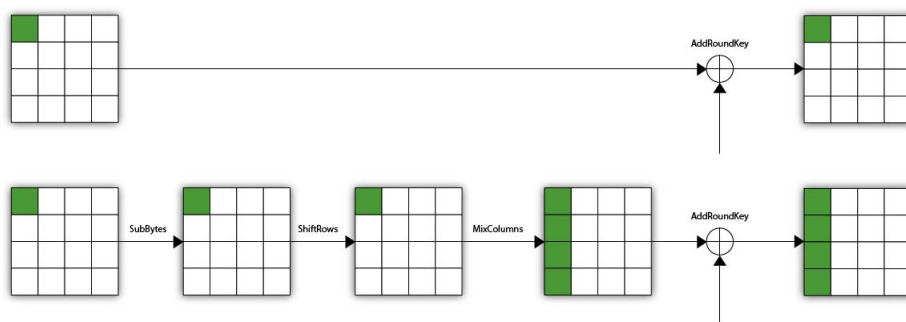
$$y = (y_0, y_1, y_2, y_3)^T$$

具体运算规则如下:

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \cdot x_0 + 3 \cdot x_1 + x_2 + x_3 \\ x_0 + 2 \cdot x_1 + 3 \cdot x_2 + x_3 \\ x_0 + x_1 + 2 \cdot x_2 + 3 \cdot x_3 \\ 3 \cdot x_0 + x_1 + x_2 + 2 \cdot x_3 \end{bmatrix}$$

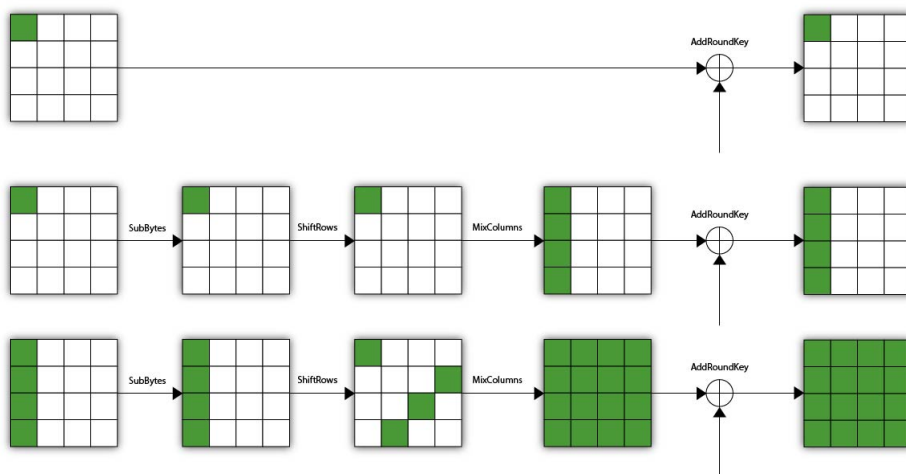
我们的输入是第一列的活跃字节  $x_0$  和其他列的常数字节。经过 MixColumns 后，第一列的输出将是所有输入字节的线性组合，只要包含  $x_0$ ，都会变成活跃字节，因为当  $x_0$  遍历  $0 \dots 255$  时，输出也会完整遍历  $0 \dots 255$ 。因此，MixColumns 不会破坏活跃字节的性质，但它会将活跃字节的影响扩散到整列。

#### 2.4.6 第一轮结束结果



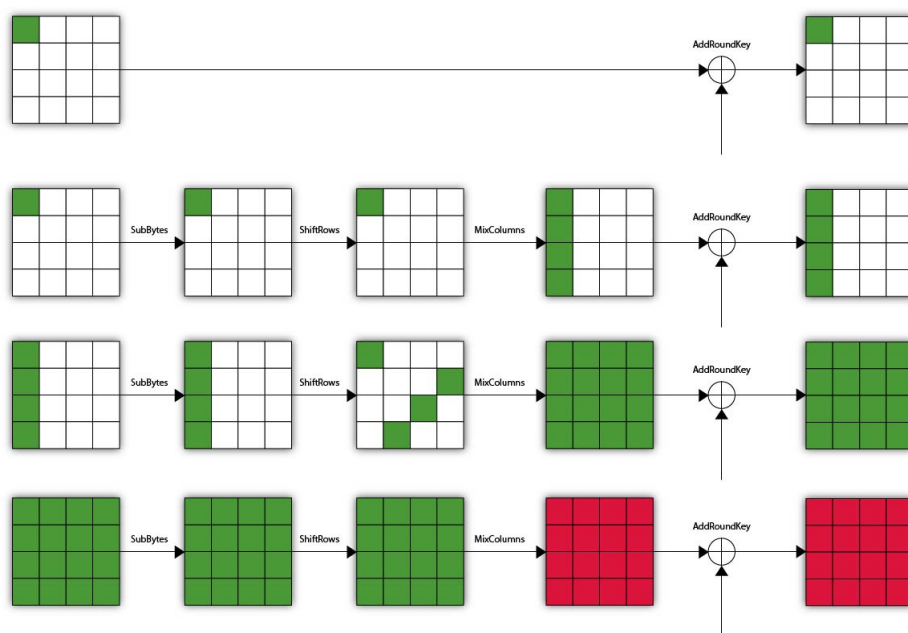
经过第一轮加密，我们得到了第一列都为活跃字节的状态。

#### 2.4.7 第二轮结束结果



接下来进入第二轮，第二轮的 SubBytes 和 ShiftRows 不会改变活跃字节的数量，但 ShiftRows 会将第一列的活跃字节扩散到其他列，然后 MixColumns 会将这些活跃字节的影响扩散到整列。最终，在第二轮结束时，状态矩阵中的所有字节都变成了活跃字节。

## 2.4.8 第三轮结束结果



到了第三轮发生了变化，在 MixColumns 之后，活跃字节的线性组合，就不一定还是活跃字节了，因为  $y_i$  的取值范围可能被限制在一个子集内，导致无法遍历完整的  $0 \dots 255$ 。这一点由实验结果验证了，比如  $y_0$  的取值大概只有 160-170 个

```
(.venv) infinite@infiniteMacBook-Air ~/Documents /Users/infinite/Documents/.venv/bin/pyt
Users/infinite/Documents/crypto_introduction/first_experiment/test.py
字节 y0 的唯一值个数: 163
字节 y0 的异或和: 0
(.venv) infinite@infiniteMacBook-Air ~/Documents /Users/infinite/Documents/.venv/bin/pyt
Users/infinite/Documents/crypto_introduction/first_experiment/test.py
字节 y0 的唯一值个数: 164
字节 y0 的异或和: 0
(.venv) infinite@infiniteMacBook-Air ~/Documents /Users/infinite/Documents/.venv/bin/pyt
Users/infinite/Documents/crypto_introduction/first_experiment/test.py
字节 y0 的唯一值个数: 156
字节 y0 的异或和: 0
(.venv) infinite@infiniteMacBook-Air ~/Documents /Users/infinite/Documents/.venv/bin/pyt
Users/infinite/Documents/crypto_introduction/first_experiment/test.py
字节 y0 的唯一值个数: 166
字节 y0 的异或和: 0
(.venv) infinite@infiniteMacBook-Air ~/Documents
```

不过仍然满足异或和为 0 的性质

下面给出证明

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} x_{i,0} \\ x_{i,1} \\ x_{i,2} \\ x_{i,3} \end{bmatrix} = \begin{bmatrix} 2 \cdot x_{i,0} + 3 \cdot x_{i,1} + x_{i,2} + x_{i,3} \\ x_{i,0} + 2 \cdot x_{i,1} + 3 \cdot x_{i,2} + x_{i,3} \\ x_{i,0} + x_{i,1} + 2 \cdot x_{i,2} + 3 \cdot x_{i,3} \\ 3 \cdot x_{i,0} + x_{i,1} + x_{i,2} + 2 \cdot x_{i,3} \end{bmatrix}$$

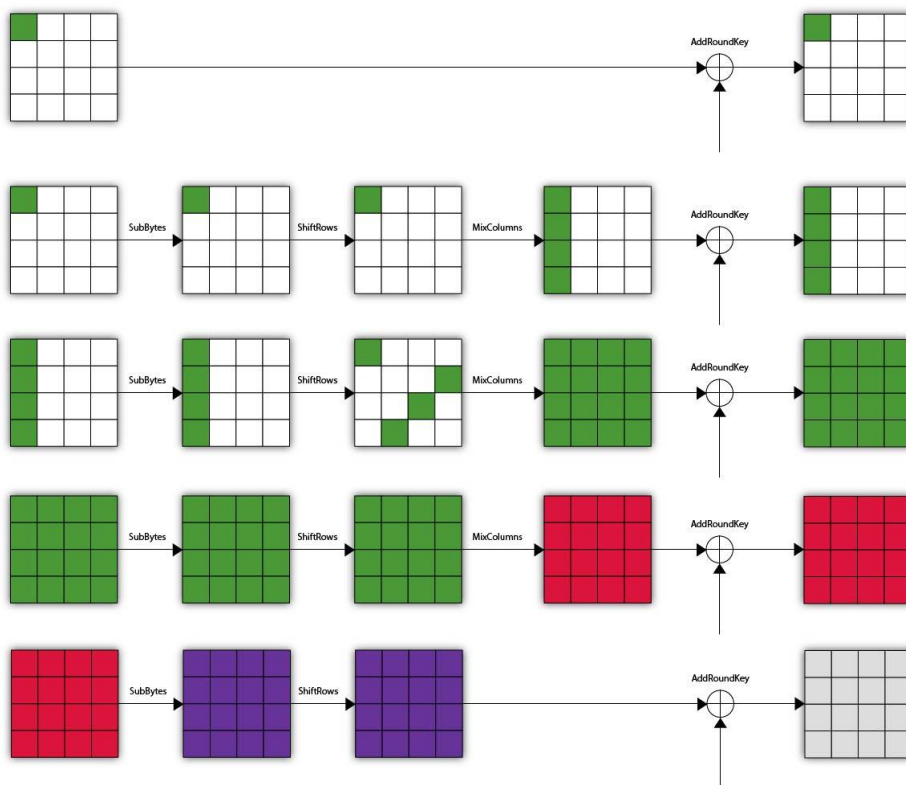
例如对于  $y_0$ ，满足

$$\bigoplus_{j=1}^{256} y_{0,j} = \bigoplus_{j=1}^{256} (2 \cdot x_{j,0} + 3 \cdot x_{j,1} + x_{j,2} + x_{j,3}) = 2 \bigoplus_{j=1}^{256} x_{j,0} \oplus 3 \bigoplus_{j=1}^{256} x_{j,1} \oplus \bigoplus_{j=1}^{256} x_{j,2} \oplus \bigoplus_{j=1}^{256} x_{j,3} = 0$$

所以在经过 3 轮 AES 加密后，其密文状态满足：

$$\bigoplus_{i=1}^{256} C^{(i)} = 0 \quad (4)$$

#### 2.4.9 四轮的情况



前面已经说明，经过三轮加密之后，密文异或和为 0，但是在第四轮的 SubBytes 操作之后，这个性质就不再成立了。不妨设经过 SubBytes 之后的  $y_0$  为  $z_0$

$$z_0 = S(y_0) = S(2 \cdot x_{i,0} + 3 \cdot x_{i,1} + x_{i,2} + x_{i,3})$$

由于过 S 盒是一个非线性变换，所以不能像第三轮那样拆开来异或运算了，因此无法保证异或和为 0

```

字节 z0 的异或和: 153
异或和不为 0"
(.venv) infinite@infiniteMacBook-Air ~/Documents /Users/infinite/Documents/.venv/bin/python
Users/infinite/Documents/crypto_introduction/first_experiment/report/code/test.py
字节 y0 的唯一值个数: 153
字节 y0 的异或和: 0
第 4 轮 SubBytes 输出
字节 z0 的唯一值个数: 153
字节 z0 的异或和: 156
异或和不为 0
(.venv) infinite@infiniteMacBook-Air ~/Documents

```

#### 2.4.10 删除 MixColumns 的影响

实验发现，若删除 MixColumns，即使加密至 31 轮，平衡性依然保持。这是因为没有 MixColumns 时，每个字节的变换仅由 S 盒复合组成。由于 S 盒是置换（Permutation），

多个置换的复合依然是置换。如果输入遍历了  $0 \dots 255$ ，输出必然也完整遍历  $0 \dots 255$ 。已知：

$$\bigoplus_{i=0}^{255} i = 0 \quad (5)$$

故该现象在无扩散层时会永久保持。

## 2.5 实验结论

本实验成功验证了 AES 的积分特性。实验证明，列混淆（MixColumns）不仅提供了列内的线性扩散，更是打破代数平衡性、防止积分攻击的关键组件。没有 MixColumns 的 AES 即使轮数再多，也无法防御结构化攻击。



## .1 完整 S1 盒差分分布表 (DDT)

Table 10: DES S1 盒完整差分分布表

| Input Xor \ Output Xor | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 00                     | 64 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 01                     | 0  | 0  | 0  | 6  | 0  | 2  | 4  | 4  | 0  | 10 | 12 | 4  | 10 | 6  | 2  | 4  |
| 02                     | 0  | 0  | 0  | 0  | 8  | 0  | 4  | 4  | 4  | 0  | 6  | 8  | 6  | 12 | 6  | 4  |
| 03                     | 14 | 4  | 2  | 2  | 10 | 6  | 4  | 2  | 6  | 4  | 4  | 0  | 2  | 2  | 2  | 0  |
| 04                     | 0  | 0  | 0  | 6  | 0  | 10 | 10 | 6  | 0  | 4  | 6  | 4  | 2  | 8  | 6  | 2  |
| 05                     | 4  | 8  | 6  | 2  | 2  | 4  | 4  | 2  | 0  | 4  | 4  | 0  | 12 | 2  | 4  | 6  |
| 06                     | 0  | 4  | 2  | 4  | 8  | 2  | 6  | 2  | 8  | 4  | 4  | 2  | 4  | 2  | 0  | 12 |
| 07                     | 2  | 4  | 10 | 4  | 0  | 4  | 8  | 4  | 2  | 4  | 8  | 2  | 2  | 2  | 4  | 4  |
| 08                     | 0  | 0  | 0  | 12 | 0  | 8  | 8  | 4  | 0  | 6  | 2  | 8  | 8  | 2  | 2  | 4  |
| 09                     | 10 | 2  | 4  | 0  | 2  | 4  | 6  | 0  | 2  | 2  | 8  | 0  | 10 | 0  | 2  | 12 |
| 0A                     | 0  | 8  | 6  | 2  | 2  | 8  | 6  | 0  | 6  | 4  | 6  | 0  | 4  | 0  | 2  | 10 |
| 0B                     | 2  | 4  | 0  | 10 | 2  | 2  | 4  | 0  | 2  | 6  | 2  | 6  | 6  | 4  | 2  | 12 |
| 0C                     | 0  | 0  | 0  | 8  | 0  | 6  | 6  | 0  | 0  | 6  | 6  | 4  | 6  | 6  | 14 | 2  |
| 0D                     | 6  | 6  | 4  | 8  | 4  | 8  | 2  | 6  | 0  | 6  | 4  | 6  | 0  | 2  | 0  | 2  |
| 0E                     | 0  | 4  | 8  | 8  | 6  | 6  | 4  | 0  | 6  | 6  | 4  | 0  | 0  | 4  | 0  | 8  |
| 0F                     | 2  | 0  | 2  | 4  | 4  | 6  | 4  | 2  | 4  | 8  | 2  | 2  | 2  | 6  | 8  | 8  |
| 10                     | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 14 | 0  | 6  | 6  | 12 | 4  | 6  | 8  | 6  |
| 11                     | 6  | 8  | 2  | 4  | 6  | 4  | 8  | 6  | 4  | 0  | 6  | 6  | 0  | 4  | 0  | 0  |
| 12                     | 0  | 8  | 4  | 2  | 6  | 6  | 4  | 6  | 6  | 4  | 2  | 6  | 6  | 0  | 4  | 0  |
| 13                     | 2  | 4  | 4  | 6  | 2  | 0  | 4  | 6  | 2  | 0  | 6  | 8  | 4  | 6  | 4  | 6  |
| 14                     | 0  | 8  | 8  | 0  | 10 | 0  | 4  | 2  | 8  | 2  | 2  | 4  | 4  | 8  | 4  | 0  |
| 15                     | 0  | 4  | 6  | 4  | 2  | 2  | 4  | 10 | 6  | 2  | 0  | 10 | 0  | 4  | 6  | 4  |
| 16                     | 0  | 8  | 10 | 8  | 0  | 2  | 2  | 6  | 10 | 2  | 0  | 2  | 0  | 6  | 2  | 6  |
| 17                     | 4  | 4  | 6  | 0  | 10 | 6  | 0  | 2  | 4  | 4  | 4  | 6  | 6  | 6  | 2  | 0  |
| 18                     | 0  | 6  | 6  | 0  | 8  | 4  | 2  | 2  | 2  | 4  | 6  | 8  | 6  | 6  | 2  | 2  |
| 19                     | 2  | 6  | 2  | 4  | 0  | 8  | 4  | 6  | 10 | 4  | 0  | 4  | 2  | 8  | 4  | 0  |
| 1A                     | 0  | 6  | 4  | 0  | 4  | 6  | 6  | 6  | 6  | 2  | 2  | 0  | 4  | 4  | 6  | 8  |
| 1B                     | 4  | 4  | 2  | 4  | 10 | 6  | 6  | 4  | 6  | 2  | 2  | 4  | 2  | 2  | 4  | 2  |
| 1C                     | 0  | 10 | 10 | 6  | 6  | 0  | 0  | 12 | 6  | 4  | 0  | 0  | 2  | 4  | 4  | 0  |
| 1D                     | 4  | 2  | 4  | 0  | 8  | 0  | 0  | 2  | 10 | 0  | 2  | 6  | 6  | 6  | 14 | 0  |
| 1E                     | 0  | 2  | 6  | 0  | 14 | 2  | 0  | 0  | 6  | 4  | 10 | 8  | 2  | 2  | 6  | 2  |
| 1F                     | 2  | 4  | 10 | 6  | 2  | 2  | 2  | 8  | 6  | 8  | 0  | 0  | 0  | 4  | 6  | 4  |
| 20                     | 0  | 0  | 0  | 10 | 0  | 12 | 8  | 2  | 0  | 6  | 4  | 4  | 4  | 2  | 0  | 12 |
| 21                     | 0  | 4  | 2  | 4  | 4  | 8  | 10 | 0  | 4  | 4  | 10 | 0  | 4  | 0  | 2  | 8  |
| 22                     | 10 | 4  | 6  | 2  | 2  | 8  | 2  | 2  | 2  | 2  | 6  | 0  | 4  | 0  | 4  | 10 |
| 23                     | 0  | 4  | 4  | 8  | 0  | 2  | 6  | 0  | 6  | 6  | 2  | 10 | 2  | 4  | 0  | 10 |
| 24                     | 12 | 0  | 0  | 2  | 2  | 2  | 2  | 0  | 14 | 14 | 2  | 0  | 2  | 6  | 2  | 4  |
| 25                     | 6  | 4  | 4  | 12 | 4  | 4  | 4  | 10 | 2  | 2  | 2  | 0  | 4  | 2  | 2  | 2  |
| 26                     | 0  | 0  | 4  | 10 | 10 | 10 | 2  | 4  | 0  | 4  | 6  | 4  | 4  | 4  | 2  | 0  |
| 27                     | 10 | 4  | 2  | 0  | 2  | 4  | 2  | 0  | 4  | 8  | 0  | 4  | 8  | 8  | 4  | 4  |
| 28                     | 12 | 2  | 2  | 8  | 2  | 6  | 12 | 0  | 0  | 2  | 6  | 0  | 4  | 0  | 6  | 2  |
| 29                     | 4  | 2  | 2  | 10 | 0  | 2  | 4  | 0  | 0  | 14 | 10 | 2  | 4  | 6  | 0  | 4  |
| 2A                     | 4  | 2  | 4  | 6  | 0  | 2  | 8  | 2  | 2  | 14 | 2  | 6  | 2  | 6  | 2  | 2  |

| Input Xor \ Output Xor | 0  | 1  | 2  | 3  | 4  | 5 | 6  | 7  | 8  | 9 | A | B  | C  | D  | E  | F  |
|------------------------|----|----|----|----|----|---|----|----|----|---|---|----|----|----|----|----|
| 2B                     | 12 | 2  | 2  | 2  | 4  | 6 | 6  | 2  | 0  | 2 | 6 | 2  | 6  | 0  | 8  | 4  |
| 2C                     | 4  | 2  | 2  | 4  | 0  | 2 | 10 | 4  | 2  | 2 | 4 | 8  | 8  | 4  | 2  | 6  |
| 2D                     | 6  | 2  | 6  | 2  | 8  | 4 | 4  | 4  | 2  | 4 | 6 | 0  | 8  | 2  | 0  | 6  |
| 2E                     | 6  | 6  | 2  | 2  | 0  | 2 | 4  | 6  | 4  | 0 | 6 | 2  | 12 | 2  | 6  | 4  |
| 2F                     | 2  | 2  | 2  | 2  | 2  | 6 | 8  | 8  | 2  | 4 | 4 | 6  | 8  | 2  | 4  | 2  |
| 30                     | 0  | 4  | 6  | 0  | 12 | 6 | 2  | 2  | 8  | 2 | 4 | 4  | 6  | 2  | 2  | 4  |
| 31                     | 4  | 8  | 2  | 10 | 2  | 2 | 2  | 2  | 6  | 0 | 0 | 2  | 2  | 4  | 10 | 8  |
| 32                     | 4  | 2  | 6  | 4  | 4  | 2 | 2  | 4  | 6  | 6 | 4 | 8  | 2  | 2  | 8  | 0  |
| 33                     | 4  | 4  | 6  | 2  | 10 | 8 | 4  | 2  | 4  | 0 | 2 | 2  | 4  | 6  | 2  | 4  |
| 34                     | 0  | 8  | 16 | 6  | 2  | 0 | 0  | 12 | 6  | 0 | 0 | 0  | 0  | 8  | 0  | 6  |
| 35                     | 2  | 2  | 4  | 0  | 8  | 0 | 0  | 0  | 14 | 4 | 6 | 8  | 0  | 2  | 14 | 0  |
| 36                     | 2  | 6  | 2  | 2  | 8  | 0 | 2  | 2  | 4  | 2 | 6 | 8  | 6  | 4  | 10 | 0  |
| 37                     | 2  | 2  | 12 | 4  | 2  | 4 | 4  | 10 | 4  | 4 | 2 | 6  | 0  | 2  | 2  | 4  |
| 38                     | 0  | 6  | 2  | 2  | 2  | 0 | 2  | 2  | 4  | 6 | 4 | 4  | 4  | 6  | 10 | 10 |
| 39                     | 6  | 2  | 2  | 4  | 12 | 6 | 4  | 8  | 4  | 0 | 2 | 4  | 2  | 4  | 4  | 0  |
| 3A                     | 6  | 4  | 6  | 4  | 6  | 8 | 0  | 6  | 2  | 2 | 6 | 2  | 2  | 6  | 4  | 0  |
| 3B                     | 2  | 6  | 4  | 0  | 0  | 2 | 4  | 6  | 4  | 6 | 8 | 6  | 4  | 4  | 6  | 2  |
| 3C                     | 0  | 10 | 4  | 0  | 12 | 0 | 4  | 2  | 6  | 0 | 4 | 12 | 4  | 4  | 2  | 0  |
| 3D                     | 0  | 8  | 6  | 2  | 2  | 6 | 0  | 8  | 4  | 4 | 0 | 4  | 0  | 12 | 4  | 4  |
| 3E                     | 4  | 8  | 2  | 2  | 2  | 4 | 4  | 14 | 4  | 2 | 0 | 2  | 0  | 8  | 4  | 4  |
| 3F                     | 4  | 8  | 4  | 2  | 4  | 0 | 2  | 4  | 4  | 2 | 4 | 8  | 8  | 6  | 2  | 2  |

## .2 完整 S2 盒差分分布表 (DDT)

Table 11: DES S2 盒完整差分分布表

| Input Xor \ Output Xor | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F |
|------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|
| 00                     | 64 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0 |
| 01                     | 0  | 0  | 0  | 4  | 0  | 2  | 6  | 4  | 0  | 14 | 8  | 6  | 8  | 4  | 6  | 2 |
| 02                     | 0  | 0  | 0  | 2  | 0  | 4  | 6  | 4  | 0  | 0  | 4  | 6  | 10 | 10 | 12 | 6 |
| 03                     | 4  | 8  | 4  | 8  | 4  | 6  | 4  | 2  | 4  | 2  | 2  | 4  | 6  | 2  | 0  | 4 |
| 04                     | 0  | 0  | 0  | 0  | 0  | 6  | 0  | 14 | 0  | 6  | 10 | 4  | 10 | 6  | 4  | 4 |
| 05                     | 2  | 0  | 4  | 8  | 2  | 4  | 6  | 6  | 2  | 0  | 8  | 4  | 2  | 4  | 10 | 2 |
| 06                     | 0  | 12 | 6  | 4  | 6  | 4  | 6  | 2  | 2  | 10 | 2  | 8  | 2  | 0  | 0  | 0 |
| 07                     | 4  | 6  | 6  | 4  | 2  | 4  | 4  | 2  | 6  | 4  | 2  | 4  | 4  | 6  | 0  | 6 |
| 08                     | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 8  | 0  | 10 | 16 | 6  | 6  | 0  | 6  | 4 |
| 09                     | 14 | 2  | 4  | 10 | 2  | 8  | 2  | 6  | 2  | 4  | 0  | 0  | 2  | 2  | 2  | 4 |
| 0A                     | 0  | 6  | 6  | 2  | 10 | 4  | 10 | 2  | 6  | 2  | 2  | 4  | 2  | 2  | 4  | 2 |
| 0B                     | 6  | 2  | 2  | 0  | 2  | 4  | 6  | 2  | 10 | 2  | 0  | 6  | 6  | 4  | 4  | 8 |
| 0C                     | 0  | 0  | 0  | 4  | 0  | 14 | 0  | 10 | 0  | 6  | 2  | 4  | 4  | 8  | 6  | 6 |
| 0D                     | 6  | 2  | 6  | 2  | 10 | 2  | 0  | 4  | 0  | 10 | 4  | 2  | 8  | 2  | 2  | 4 |
| 0E                     | 0  | 6  | 12 | 8  | 0  | 4  | 2  | 0  | 8  | 2  | 4  | 4  | 6  | 2  | 0  | 6 |
| 0F                     | 0  | 8  | 2  | 0  | 6  | 6  | 8  | 2  | 4  | 4  | 4  | 6  | 8  | 0  | 4  | 2 |
| 10                     | 0  | 0  | 0  | 8  | 0  | 4  | 10 | 2  | 0  | 2  | 8  | 10 | 0  | 10 | 6  | 4 |

| Input Xor \ Output Xor | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 11                     | 6  | 6  | 4  | 6  | 4  | 0  | 6  | 4  | 8  | 2  | 10 | 2  | 2  | 4  | 0  | 0  |
| 12                     | 0  | 6  | 2  | 6  | 2  | 4  | 12 | 4  | 6  | 4  | 0  | 4  | 4  | 6  | 2  | 2  |
| 13                     | 4  | 0  | 4  | 0  | 8  | 6  | 6  | 0  | 0  | 2  | 0  | 6  | 4  | 8  | 2  | 14 |
| 14                     | 0  | 6  | 6  | 4  | 10 | 0  | 2  | 12 | 6  | 2  | 2  | 2  | 4  | 4  | 2  | 2  |
| 15                     | 6  | 8  | 2  | 0  | 8  | 2  | 0  | 2  | 2  | 2  | 2  | 2  | 2  | 14 | 10 | 2  |
| 16                     | 0  | 8  | 6  | 4  | 2  | 2  | 4  | 2  | 6  | 4  | 6  | 2  | 6  | 0  | 6  | 6  |
| 17                     | 6  | 4  | 8  | 6  | 4  | 4  | 0  | 4  | 6  | 2  | 4  | 4  | 4  | 2  | 4  | 2  |
| 18                     | 0  | 6  | 4  | 6  | 10 | 4  | 0  | 2  | 4  | 8  | 0  | 0  | 4  | 8  | 2  | 6  |
| 19                     | 2  | 4  | 6  | 4  | 4  | 2  | 4  | 2  | 6  | 4  | 6  | 8  | 0  | 6  | 4  | 2  |
| 1A                     | 0  | 6  | 8  | 4  | 2  | 4  | 2  | 2  | 8  | 2  | 2  | 6  | 2  | 4  | 4  | 8  |
| 1B                     | 0  | 6  | 4  | 4  | 0  | 12 | 6  | 4  | 2  | 2  | 2  | 4  | 4  | 2  | 10 | 2  |
| 1C                     | 0  | 4  | 6  | 6  | 12 | 0  | 4  | 0  | 10 | 2  | 6  | 2  | 0  | 0  | 10 | 2  |
| 1D                     | 0  | 6  | 2  | 2  | 6  | 0  | 4  | 16 | 4  | 4  | 2  | 0  | 0  | 4  | 6  | 8  |
| 1E                     | 0  | 4  | 8  | 2  | 10 | 6  | 6  | 0  | 8  | 4  | 0  | 2  | 4  | 4  | 0  | 6  |
| 1F                     | 4  | 2  | 6  | 6  | 2  | 2  | 2  | 4  | 8  | 6  | 10 | 6  | 4  | 0  | 0  | 2  |
| 20                     | 0  | 0  | 0  | 2  | 0  | 12 | 10 | 4  | 0  | 0  | 0  | 2  | 14 | 2  | 8  | 10 |
| 21                     | 0  | 4  | 6  | 8  | 2  | 10 | 4  | 2  | 2  | 6  | 4  | 2  | 6  | 2  | 0  | 6  |
| 22                     | 4  | 12 | 8  | 4  | 2  | 2  | 0  | 0  | 2  | 8  | 8  | 6  | 0  | 6  | 0  | 2  |
| 23                     | 8  | 2  | 0  | 2  | 8  | 4  | 2  | 6  | 4  | 8  | 2  | 2  | 6  | 4  | 2  | 4  |
| 24                     | 10 | 4  | 0  | 0  | 0  | 4  | 0  | 2  | 6  | 8  | 6  | 10 | 8  | 0  | 2  | 4  |
| 25                     | 6  | 0  | 12 | 2  | 8  | 6  | 10 | 0  | 0  | 8  | 2  | 6  | 0  | 0  | 2  | 2  |
| 26                     | 2  | 2  | 4  | 4  | 2  | 2  | 10 | 14 | 2  | 0  | 4  | 2  | 2  | 4  | 6  | 4  |
| 27                     | 6  | 0  | 0  | 2  | 6  | 4  | 2  | 4  | 4  | 4  | 8  | 4  | 8  | 0  | 6  | 6  |
| 28                     | 8  | 0  | 8  | 2  | 4  | 12 | 2  | 0  | 2  | 6  | 2  | 0  | 6  | 2  | 0  | 10 |
| 29                     | 0  | 2  | 4  | 10 | 2  | 8  | 6  | 4  | 0  | 10 | 0  | 2  | 10 | 0  | 2  | 4  |
| 2A                     | 4  | 0  | 4  | 8  | 6  | 2  | 4  | 4  | 6  | 6  | 2  | 6  | 2  | 2  | 4  | 4  |
| 2B                     | 2  | 2  | 6  | 4  | 0  | 2  | 2  | 6  | 2  | 8  | 8  | 4  | 4  | 4  | 8  | 2  |
| 2C                     | 10 | 6  | 8  | 6  | 0  | 6  | 4  | 4  | 4  | 2  | 4  | 4  | 0  | 0  | 2  | 4  |
| 2D                     | 2  | 2  | 2  | 4  | 0  | 0  | 0  | 2  | 8  | 4  | 4  | 6  | 10 | 2  | 14 | 4  |
| 2E                     | 2  | 4  | 0  | 2  | 10 | 4  | 2  | 0  | 2  | 2  | 6  | 2  | 8  | 8  | 10 | 2  |
| 2F                     | 12 | 4  | 6  | 8  | 2  | 6  | 2  | 8  | 0  | 4  | 0  | 2  | 0  | 8  | 2  | 0  |
| 30                     | 0  | 4  | 0  | 2  | 4  | 4  | 8  | 6  | 10 | 6  | 2  | 12 | 0  | 0  | 0  | 6  |
| 31                     | 0  | 10 | 2  | 0  | 6  | 2  | 10 | 2  | 6  | 0  | 2  | 0  | 6  | 6  | 4  | 8  |
| 32                     | 8  | 4  | 6  | 0  | 6  | 4  | 4  | 8  | 4  | 6  | 8  | 0  | 2  | 2  | 2  | 0  |
| 33                     | 2  | 2  | 6  | 10 | 2  | 0  | 0  | 6  | 4  | 4  | 12 | 8  | 4  | 2  | 2  | 0  |
| 34                     | 0  | 12 | 6  | 4  | 6  | 0  | 4  | 4  | 4  | 0  | 4  | 6  | 4  | 2  | 4  | 4  |
| 35                     | 0  | 12 | 4  | 6  | 2  | 4  | 4  | 0  | 10 | 0  | 0  | 8  | 0  | 8  | 0  | 6  |
| 36                     | 8  | 2  | 4  | 0  | 4  | 0  | 4  | 2  | 0  | 8  | 4  | 2  | 6  | 16 | 2  | 2  |
| 37                     | 6  | 2  | 2  | 2  | 6  | 6  | 4  | 8  | 2  | 2  | 6  | 2  | 2  | 2  | 4  | 8  |
| 38                     | 0  | 8  | 8  | 10 | 6  | 2  | 2  | 0  | 4  | 0  | 4  | 2  | 4  | 0  | 4  | 10 |
| 39                     | 0  | 2  | 0  | 0  | 8  | 0  | 10 | 4  | 10 | 0  | 8  | 4  | 4  | 4  | 4  | 6  |
| 3A                     | 4  | 0  | 2  | 8  | 4  | 2  | 2  | 2  | 4  | 8  | 2  | 0  | 4  | 10 | 10 | 2  |
| 3B                     | 16 | 4  | 4  | 2  | 8  | 2  | 2  | 6  | 4  | 4  | 4  | 2  | 0  | 2  | 2  | 2  |
| 3C                     | 0  | 2  | 6  | 2  | 8  | 4  | 6  | 0  | 10 | 2  | 2  | 4  | 4  | 10 | 4  | 0  |
| 3D                     | 0  | 16 | 10 | 2  | 4  | 2  | 4  | 2  | 8  | 0  | 0  | 8  | 0  | 6  | 2  | 0  |
| 3E                     | 4  | 4  | 0  | 10 | 2  | 4  | 2  | 14 | 4  | 2  | 6  | 6  | 0  | 0  | 6  | 0  |
| 3F                     | 4  | 0  | 0  | 2  | 0  | 8  | 2  | 4  | 0  | 2  | 4  | 4  | 4  | 14 | 10 | 6  |

### .3 完整 S3 盒差分分布表 (DDT)

Table 12: DES S3 盒完整差分分布表

| Input Xor \ Output Xor | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 00                     | 64 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 01                     | 0  | 0  | 0  | 2  | 0  | 4  | 2  | 12 | 0  | 14 | 0  | 4  | 8  | 2  | 6  | 10 |
| 02                     | 0  | 0  | 0  | 2  | 0  | 2  | 0  | 8  | 0  | 4  | 12 | 10 | 4  | 6  | 8  | 8  |
| 03                     | 8  | 6  | 10 | 4  | 8  | 6  | 0  | 6  | 4  | 4  | 0  | 0  | 0  | 4  | 2  | 2  |
| 04                     | 0  | 0  | 0  | 4  | 0  | 2  | 4  | 2  | 0  | 12 | 8  | 4  | 6  | 8  | 10 | 4  |
| 05                     | 6  | 2  | 4  | 8  | 6  | 10 | 6  | 2  | 2  | 8  | 2  | 0  | 2  | 0  | 4  | 2  |
| 06                     | 0  | 10 | 6  | 6  | 10 | 0  | 4  | 12 | 2  | 4  | 0  | 0  | 6  | 4  | 0  | 0  |
| 07                     | 2  | 0  | 0  | 4  | 4  | 4  | 4  | 2  | 10 | 4  | 4  | 8  | 4  | 4  | 4  | 6  |
| 08                     | 0  | 0  | 0  | 10 | 0  | 4  | 4  | 6  | 0  | 6  | 6  | 6  | 6  | 0  | 8  | 8  |
| 09                     | 10 | 2  | 0  | 2  | 10 | 4  | 6  | 2  | 0  | 6  | 0  | 4  | 6  | 2  | 4  | 6  |
| 0A                     | 0  | 10 | 6  | 0  | 14 | 6  | 4  | 0  | 4  | 6  | 6  | 0  | 4  | 0  | 2  | 2  |
| 0B                     | 2  | 6  | 2  | 10 | 2  | 2  | 4  | 0  | 4  | 2  | 6  | 0  | 2  | 8  | 14 | 0  |
| 0C                     | 0  | 0  | 0  | 8  | 0  | 12 | 12 | 4  | 0  | 8  | 0  | 4  | 2  | 10 | 2  | 2  |
| 0D                     | 8  | 2  | 8  | 0  | 0  | 4  | 2  | 0  | 2  | 8  | 14 | 2  | 6  | 2  | 4  | 2  |
| 0E                     | 0  | 4  | 4  | 2  | 4  | 2  | 4  | 4  | 10 | 4  | 4  | 4  | 4  | 4  | 2  | 8  |
| 0F                     | 4  | 6  | 4  | 6  | 2  | 2  | 4  | 8  | 6  | 2  | 6  | 2  | 0  | 6  | 2  | 4  |
| 10                     | 0  | 0  | 0  | 4  | 0  | 12 | 4  | 8  | 0  | 4  | 2  | 6  | 2  | 14 | 0  | 8  |
| 11                     | 8  | 2  | 2  | 6  | 4  | 0  | 2  | 0  | 8  | 4  | 12 | 2  | 10 | 0  | 2  | 2  |
| 12                     | 0  | 2  | 8  | 2  | 4  | 8  | 0  | 8  | 8  | 0  | 2  | 2  | 4  | 2  | 14 | 0  |
| 13                     | 4  | 4  | 12 | 0  | 2  | 2  | 2  | 10 | 2  | 2  | 2  | 2  | 4  | 4  | 4  | 8  |
| 14                     | 0  | 6  | 4  | 4  | 6  | 4  | 6  | 2  | 8  | 6  | 6  | 2  | 2  | 0  | 0  | 8  |
| 15                     | 4  | 8  | 2  | 8  | 2  | 4  | 8  | 0  | 4  | 2  | 2  | 2  | 2  | 6  | 8  | 2  |
| 16                     | 0  | 6  | 10 | 2  | 8  | 4  | 2  | 0  | 2  | 2  | 2  | 8  | 4  | 6  | 4  | 4  |
| 17                     | 0  | 6  | 6  | 0  | 6  | 2  | 4  | 4  | 6  | 2  | 2  | 10 | 6  | 8  | 2  | 0  |
| 18                     | 0  | 8  | 4  | 6  | 6  | 0  | 6  | 2  | 4  | 0  | 4  | 2  | 10 | 0  | 6  | 6  |
| 19                     | 4  | 2  | 4  | 8  | 4  | 2  | 10 | 2  | 2  | 2  | 6  | 8  | 2  | 6  | 0  | 2  |
| 1A                     | 0  | 8  | 6  | 4  | 4  | 0  | 6  | 4  | 4  | 8  | 0  | 10 | 2  | 2  | 2  | 4  |
| 1B                     | 4  | 10 | 2  | 0  | 2  | 4  | 2  | 4  | 8  | 2  | 2  | 8  | 4  | 2  | 8  | 2  |
| 1C                     | 0  | 6  | 8  | 8  | 4  | 2  | 8  | 0  | 12 | 0  | 10 | 0  | 4  | 0  | 2  | 0  |
| 1D                     | 0  | 2  | 0  | 6  | 2  | 8  | 4  | 6  | 2  | 0  | 4  | 2  | 4  | 10 | 0  | 14 |
| 1E                     | 0  | 4  | 8  | 2  | 4  | 6  | 0  | 4  | 10 | 0  | 2  | 6  | 4  | 8  | 4  | 2  |
| 1F                     | 0  | 6  | 8  | 0  | 10 | 6  | 4  | 6  | 4  | 2  | 2  | 10 | 4  | 0  | 0  | 2  |
| 20                     | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 8  | 0  | 2  | 2  | 4  | 10 | 16 | 12 | 2  |
| 21                     | 10 | 8  | 8  | 0  | 8  | 4  | 2  | 4  | 0  | 6  | 6  | 6  | 0  | 0  | 2  | 0  |
| 22                     | 12 | 6  | 4  | 4  | 2  | 4  | 10 | 2  | 0  | 4  | 4  | 2  | 4  | 4  | 0  | 2  |
| 23                     | 2  | 2  | 0  | 6  | 0  | 2  | 4  | 0  | 4  | 12 | 4  | 2  | 6  | 4  | 8  | 8  |
| 24                     | 4  | 8  | 2  | 12 | 6  | 4  | 2  | 10 | 2  | 2  | 2  | 4  | 2  | 0  | 4  | 0  |
| 25                     | 6  | 0  | 2  | 0  | 8  | 2  | 0  | 2  | 8  | 8  | 2  | 2  | 4  | 4  | 10 | 6  |
| 26                     | 6  | 2  | 0  | 4  | 4  | 0  | 4  | 0  | 4  | 2  | 14 | 0  | 8  | 10 | 0  | 6  |
| 27                     | 0  | 2  | 4  | 16 | 8  | 6  | 6  | 6  | 0  | 2  | 4  | 4  | 0  | 2  | 2  | 2  |
| 28                     | 6  | 2  | 10 | 0  | 6  | 4  | 0  | 4  | 4  | 2  | 4  | 8  | 2  | 2  | 8  | 2  |

| Input Xor \ Output Xor | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A | B | C  | D  | E  | F  |
|------------------------|----|----|----|----|----|----|----|----|----|----|---|---|----|----|----|----|
| 29                     | 0  | 2  | 8  | 4  | 0  | 4  | 0  | 6  | 4  | 10 | 4 | 8 | 4  | 4  | 4  | 2  |
| 2A                     | 2  | 6  | 0  | 4  | 2  | 4  | 4  | 6  | 4  | 8  | 4 | 4 | 4  | 2  | 4  | 6  |
| 2B                     | 10 | 2  | 6  | 6  | 4  | 4  | 8  | 0  | 4  | 2  | 2 | 0 | 2  | 4  | 4  | 6  |
| 2C                     | 10 | 4  | 6  | 2  | 4  | 2  | 2  | 2  | 4  | 10 | 4 | 4 | 0  | 2  | 6  | 2  |
| 2D                     | 4  | 2  | 4  | 4  | 4  | 2  | 4  | 16 | 2  | 0  | 0 | 4 | 4  | 2  | 6  | 6  |
| 2E                     | 4  | 0  | 2  | 10 | 0  | 6  | 10 | 4  | 2  | 6  | 6 | 2 | 2  | 0  | 2  | 8  |
| 2F                     | 8  | 2  | 0  | 0  | 4  | 4  | 4  | 2  | 6  | 4  | 6 | 2 | 4  | 8  | 4  | 6  |
| 30                     | 0  | 10 | 8  | 6  | 2  | 0  | 4  | 2  | 10 | 4  | 4 | 6 | 2  | 0  | 6  | 0  |
| 31                     | 2  | 6  | 2  | 0  | 4  | 2  | 8  | 8  | 2  | 2  | 2 | 0 | 2  | 12 | 6  | 6  |
| 32                     | 2  | 0  | 4  | 8  | 2  | 8  | 4  | 4  | 8  | 4  | 2 | 8 | 6  | 2  | 0  | 2  |
| 33                     | 4  | 4  | 6  | 8  | 6  | 6  | 0  | 2  | 2  | 2  | 6 | 4 | 12 | 0  | 0  | 2  |
| 34                     | 0  | 6  | 2  | 2  | 16 | 2  | 2  | 2  | 12 | 2  | 4 | 0 | 4  | 2  | 0  | 8  |
| 35                     | 4  | 6  | 0  | 10 | 8  | 0  | 2  | 2  | 6  | 0  | 0 | 6 | 2  | 10 | 2  | 6  |
| 36                     | 4  | 4  | 4  | 4  | 0  | 6  | 6  | 4  | 4  | 4  | 4 | 4 | 0  | 6  | 2  | 8  |
| 37                     | 4  | 8  | 2  | 4  | 2  | 2  | 6  | 0  | 2  | 4  | 8 | 4 | 10 | 0  | 6  | 2  |
| 38                     | 0  | 8  | 12 | 0  | 2  | 2  | 6  | 6  | 2  | 10 | 2 | 2 | 0  | 8  | 0  | 4  |
| 39                     | 2  | 6  | 4  | 0  | 6  | 4  | 6  | 4  | 8  | 0  | 4 | 4 | 2  | 4  | 8  | 2  |
| 3A                     | 6  | 0  | 2  | 2  | 4  | 6  | 4  | 4  | 4  | 2  | 2 | 6 | 12 | 2  | 6  | 2  |
| 3B                     | 2  | 2  | 6  | 0  | 0  | 10 | 4  | 8  | 4  | 2  | 4 | 8 | 4  | 4  | 0  | 6  |
| 3C                     | 0  | 2  | 4  | 2  | 12 | 2  | 0  | 6  | 2  | 0  | 2 | 8 | 4  | 6  | 4  | 10 |
| 3D                     | 4  | 6  | 8  | 6  | 2  | 2  | 2  | 2  | 10 | 2  | 6 | 6 | 2  | 4  | 2  | 0  |
| 3E                     | 8  | 6  | 4  | 4  | 2  | 10 | 2  | 0  | 2  | 2  | 4 | 2 | 4  | 2  | 10 | 2  |
| 3F                     | 2  | 6  | 4  | 0  | 0  | 10 | 8  | 2  | 2  | 8  | 6 | 4 | 6  | 2  | 0  | 4  |

## .4 实验完整代码

### .4.1 实验一代码

```
# DES S boxes
S1 = [
    [14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7],
    [0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8],
    [4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0],
    [15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13]
]

S2 = [
    [15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10],
    [3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5],
    [0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15],
    [13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9]
]

S3 = [
    [10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8],

```

```
[13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1],
[13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7],
[1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12]
]

def sbbox_lookup(x, sbbox):
    # x is 6-bit integer
    row = ((x & 0b100000) >> 4) + (x & 0b000001)
    col = (x >> 1) & 0b1111
    return sbbox[row][col]

def compute_ddt(sbox):
    ddt = [[0 for _ in range(16)] for _ in range(64)]
    for dx in range(64):
        for x in range(64):
            y1 = sbbox_lookup(x, sbox)
            y2 = sbbox_lookup(x ^ dx, sbox)
            dy = y1 ^ y2
            ddt[dx][dy] += 1
    return ddt

def print_ddt(ddt):
    print("      ", end="")
    for dy in range(16):
        print(f"{dy:>3}", end=" ")
    print()
    for dx in range(64):
        print(f"{dx:>3}: ", end="")
        for dy in range(16):
            print(f"{ddt[dx][dy]:>3}", end=" ")
        print()

def analyze_ddt(ddt):
    # 忽略 dx=0 的平凡情况
    max_count = 0
    max_pairs = []

    values = set()
    nonzero_to_zero = []

    for dx in range(1, 64):
        for dy in range(16):
            c = ddt[dx][dy]
            if c > 0:
                values.add(c / 64)

            if c > max_count:
                max_count = c
                max_pairs = [(dx, dy)]
```

```

        elif c == max_count:
            max_pairs.append((dx, dy))

        if dy == 0 and c > 0:
            nonzero_to_zero.append((dx, c))

    return max_count, max_pairs, sorted(values), nonzero_to_zero

# 处理多个S盒
sboxes = [("S1", S1), ("S2", S2), ("S3", S3)]

for name, sbox in sboxes:
    print(f"\n=====")
    print(f"分析 {name} 盒:")
    print(f"=====")

    ddt = compute_ddt(sbox)
    print_ddt(ddt)

    max_count, max_pairs, probs, nonzero_to_zero = analyze_ddt(ddt)

    print("\n最大出现次数:", max_count)
    print("对应概率:", max_count / 64)
    print("达到最大值的 (dx, dy):", max_pairs)
    print("该S盒所有可能的非零概率传播:", probs)
    print("输入差分非0而输出差分为0的情况:", nonzero_to_zero)

```

## 4.2 实验二代码

```

import random

# AES指定加密轮数实现

class AES:
    def __init__(self, m, main_k, rounds = 10):
        self.m = self._bytes2matrix(m)
        # 转换为4*4分组,类似转置
        self.m = [list(x) for x in zip(*self.m)]

        self.main_k = main_k # 主密钥

        self.rounds = rounds # 操作轮数

        self.mixMatrix = [[2,3,1,1],[1,2,3,1],[1,1,2,3],[3,1,1,2]] #
            列混合所用矩阵

        self.sbox = (

```

```

0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0
    x67, 0x2B, 0xFE, 0xD7, 0xAB, 0x76,
0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0
    xA2, 0xAF, 0x9C, 0xA4, 0x72, 0xC0,
0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0
    xE5, 0xF1, 0x71, 0xD8, 0x31, 0x15,
0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0
    x80, 0xE2, 0xEB, 0x27, 0xB2, 0x75,
0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0
    xD6, 0xB3, 0x29, 0xE3, 0x2F, 0x84,
0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0
    xBE, 0x39, 0x4A, 0x4C, 0x58, 0xCF,
0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0
    x02, 0x7F, 0x50, 0x3C, 0x9F, 0xA8,
0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0
    xDA, 0x21, 0x10, 0xFF, 0xF3, 0xD2,
0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0
    x7E, 0x3D, 0x64, 0x5D, 0x19, 0x73,
0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0
    xB8, 0x14, 0xDE, 0x5E, 0x0B, 0xDB,
0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0
    xAC, 0x62, 0x91, 0x95, 0xE4, 0x79,
0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0
    xF4, 0xEA, 0x65, 0x7A, 0xAE, 0x08,
0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0
    x74, 0x1F, 0x4B, 0xBD, 0x8B, 0x8A,
0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0
    x57, 0xB9, 0x86, 0xC1, 0x1D, 0x9E,
0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0
    x87, 0xE9, 0xCE, 0x55, 0x28, 0xDF,
0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0
    x2D, 0x0F, 0xB0, 0x54, 0xBB, 0x16,
)

```

```

def subbytes(self):
    # 每一位共8bit,前4bit指示行号,后4bit指示列号
    for i in range(4):
        for j in range(4):
            curr_m = f"{self.m[i][j]:08b}" # 转换为8位二进制
            row = int(curr_m[:4],2)
            col = int(curr_m[4:],2)
            self.m[i][j] = self.sbox[row * 16 + col]

def shiftrows(self):
    # 行移位
    for i in range(4):
        # 在第row行左移row个位置
        self.m[i] = self.m[i][i:] + self.m[i][:i]

```



```
def gmul(self,a,b):
    # 实现a,b在有限域上的乘法
    p = 0
    for _ in range(8):
        if b & 1:
            p^=a
        hi_bit = a & 0x80
        a <<= 1
        if hi_bit:
            a^= 0x1b
        a &= 0xff
        b >>= 1
    return p

def _mix_single_col(self,col):
    # x为要混合的列,返回混合后的列
    return [
        self.gmul(2,col[0]) ^ self.gmul(3,col[1]) ^ col[2] ^ col[1],
        col[0] ^ self.gmul(2,col[1]) ^ self.gmul(3,col[2]) ^ col[3],
        col[0] ^ col[1] ^ self.gmul(2,col[2]) ^ self.gmul(3,col[3]),
        self.gmul(3,col[0]) ^ col[1] ^ col[2] ^ self.gmul(2,col[3])
    ]

def mixcolumns(self):
    # 列混合
    for i in range(4):
        curr_column = [self.m[x][i] for x in range(4)] # 第i列
        mix_curr_column = self._mix_single_col(curr_column)
        # 返回本列
        self.m[0][i],self.m[1][i],self.m[2][i],self.m[3][i] =
            mix_curr_column

def addroundkeys(self,k):
    # 轮密钥相加,即逐位异或
    for i in range(4):
        for j in range(4):
            self.m[i][j] = self.m[i][j] ^ k[i][j]

def _bytes2matrix(self,text):
    # 16bytes->4*4matrix
    return [list(text[i:i+4]) for i in range(0,len(text),4)]

def _sub_word(self,word):
    # 按字节过S盒
    return [self.sbox[b] for b in word]
```

```
def _rot_word(self, word):
    # 左移一位
    return word[1:] + word[:1]

def expand_key(self):
    # 根据主密钥进行密钥拓展,生成轮密钥
    # 本处简易128位AES NR = self.rounds, Nk = 4
    main_k = self.main_k
    r_con = (
        0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40,
        0x80, 0x1B, 0x36)

    Nk = 4
    Nb = 4
    Nr = self.rounds
    words = self._bytes2matrix(main_k)
    words = [list(w) for w in words]

    # 拓展,Nr轮加密需要将主密钥拓展为Nr+1个128bit轮密钥
    for i in range(Nk, Nb * (Nr+1)):
        temp = words[i-1][:]

        if i % Nk == 0:
            # w[i] = subWord(rotWord(w[i-1])) ^ rcon[i] ^ w[4i-4]
            # rotWord:
            temp = self._rot_word(temp)
            # subWord:
            temp = self._sub_word(temp)
            # Rcon
            temp[0] ^= r_con[ i // Nk]

        # XOR
        new_word = [a ^ b for a, b in zip(words[i - Nk], temp)]
        words.append(new_word)

    # 分组为轮密钥并返回
    round_keys = [words[4*i : 4*(i+1)] for i in range(Nr+1)]
    return round_keys

def encrypt(self, is_MC = True):
    # is_MC 对比是否执行列混合
    # 实现加密
    K = self.expand_key() # 实现密钥拓展
    # 先密钥相加
    self.addroundkeys(K[0])

    # 进行轮加密,这里根据实验加密轮数,每轮四个步骤都执行
    for i in range(1, self.rounds+1):
```

```
        self.subbytes() # 过S盒
        self.shiftrows() # 行移位
        if is_MC:
            self.mixcolumns() # 列混合
            self.addroundkeys(K[i]) # 轮密钥相加

    return self.m # 此时为加密后的密文

def generate_plaintext(fixed_bytes, active_index):
    # fixed_bytes: 长度固定的list
    # active_index : 遍历位
    plaintext = []
    for i in range(256):
        pt = fixed_bytes[:]
        pt[active_index] = i
        plaintext.append(bytes(pt))

    return plaintext

def encrypt_all(plaintext, key, rounds, is_MC = True):
    # 所有明文做加密
    ciphertexts = []
    for pt in plaintext:
        aes = AES(pt, key, rounds = rounds)
        ct_matrix = aes.encrypt(is_MC=is_MC)

        # 转回16字节
        ct = []
        for col in zip(*ct_matrix):
            ct.extend(col)
        ciphertexts.append(ct)
    return ciphertexts

def xor_all(ciphertexts):
    result = [0] * 16
    for ct in ciphertexts:
        for i in range(16):
            result[i] ^= ct[i]

    return result

def main(rounds = 3, is_MC = True):
    print("="*50)
    p0 = [
        0x10, 0x22, 0x34, 0x46,
        0x58, 0x6A, 0x7C, 0x8E,
        0x91, 0xA3, 0xB5, 0xC7,
        0xD9, 0xEB, 0xFD, 0x0F
```

```

    ]
    p1 = [
        0x01, 0x13, 0x25, 0x37,
        0x49, 0x5B, 0x6D, 0x7F,
        0x80, 0x92, 0xA4, 0xB6,
        0xC8, 0xDA, 0xEC, 0xFE
    ]
    p2 = [
        0xFF, 0xEE, 0xDD, 0xCC,
        0xBB, 0xAA, 0x99, 0x88,
        0x77, 0x66, 0x55, 0x44,
        0x33, 0x22, 0x11, 0x00
    ]
    fixed_groups = [p0,p1,p2] # 三组

    Makter_Key = "0xabf266f7a189b08e5db4a56ea585ae0f"
    key = bytes.fromhex(Makter_Key.replace("0x",""))
    print(f"加密密钥为{key}")
    idx = 4 # 移动位,即(1,0)
    print(f"加密轮数{rounds},是否MC变换{is_MC}")
    for i in range(len(fixed_groups)):
        plaintexts = generate_plaintext(fixed_groups[i],idx)
        ct = encrypt_all(plaintexts,key,rounds = rounds,is_MC= is_MC)
        xor = xor_all(ct)
        print(f"第{i+1}组异或结果:{xor}")

if __name__ == "__main__":
    # main(3,True)
    # main(4,True)
    for i in range(3,11):
        main(i,False)

```

#### 4.3 验证活跃字节代码

```

import aes
import random

cipher = aes.AES(N_ROUNDS=3)
master_key = random.randbytes(16)
round_keys = cipher.expand_key(master_key)

# 构造初始 Lambda-集合 (256个矩阵)
states = []
for x in range(256):
    p = [[0]*4 for _ in range(4)]
    p[1][0] = x
    states.append(p)

```

```
# --- 第 0 轮 ---
for i in range(256):
    states[i] = cipher.add_round_key(states[i], round_keys[0])

# --- 第 1 轮 ---
for i in range(256):
    states[i] = cipher.sub_bytes(states[i])
    cipher.shift_rows(states[i])
    cipher.mix_columns(states[i])
    states[i] = cipher.add_round_key(states[i], round_keys[1])

# --- 第 2 轮 ---
for i in range(256):
    states[i] = cipher.sub_bytes(states[i])
    cipher.shift_rows(states[i])
    cipher.mix_columns(states[i])
    states[i] = cipher.add_round_key(states[i], round_keys[2])

# --- 第 3 轮 ---
for i in range(256):
    states[i] = cipher.sub_bytes(states[i])
    cipher.shift_rows(states[i])

for i in range(256):
    cipher.mix_columns(states[i])

# 统计第 3 轮 MixColumns 输出端 (y0) 的取值个数
col_output_vals = [states[i][0][0] for i in range(256)]
unique_count = len(set(col_output_vals))
print(f"字节 y0 的唯一值个数: {unique_count}")

# 验证异或平衡性
xor_sum = 0
for val in col_output_vals:
    xor_sum ^= val
print(f"字节 y0 的异或和: {xor_sum}")

z0_output_vals = []
for val in col_output_vals:
    # 模拟 S 盒 变换
    z0_output_vals.append(cipher.s_box[val])

print("第 4 轮 SubBytes 输出")
unique_count_z = len(set(z0_output_vals))
print(f"字节 z0 的唯一值个数: {unique_count_z}")

# 验证第四轮后的平衡性 (异或和)
xor_sum_z = 0
```

```
for val in z0_output_vals:
    xor_sum_z ^= val
print(f"字节 z0 的异或和: {xor_sum_z}")

if xor_sum_z != 0:
    print("异或和不为0")
```